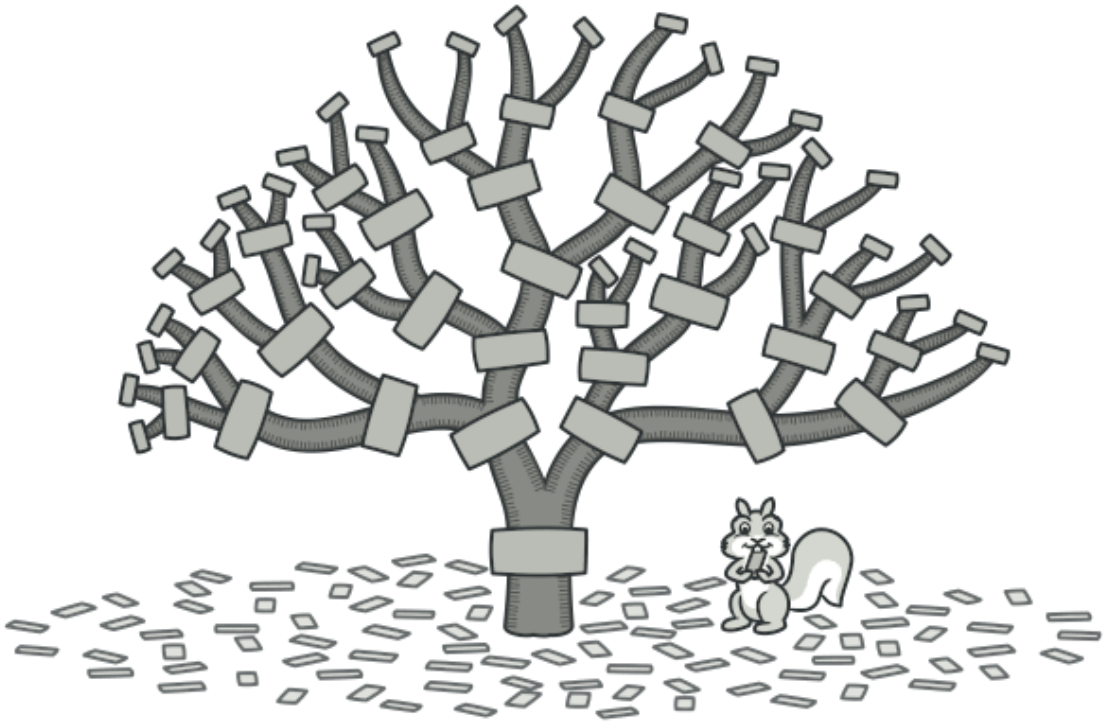
Composite

Also known as: Object Tree

Absicht

Composite ist ein strukturelles Entwurfsmuster, mit dem Sie Objekte zu Baumstrukturen zusammensetzen und dann mit diesen Strukturen arbeiten können, als wären sie einzelne Objekte.

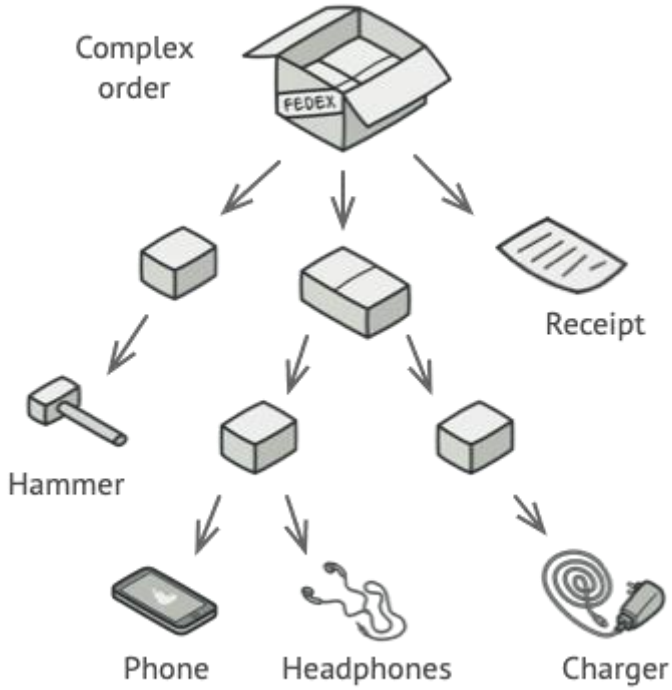


Problem

Die Verwendung des Composite-Musters ist nur dann sinnvoll, wenn das Kernmodell Ihrer App als Baum dargestellt werden kann.

Stellen Sie sich beispielsweise vor, Sie haben zwei Objekttypen: `Products`und `Boxes`. Ein `Box`kann mehrere `Products`sowie eine Anzahl kleinerer enthalten `Boxes`. Diese kleinen `Boxes`können auch einige `Products`oder sogar noch kleinere enthalten `Boxes`, und so weiter.

Angenommen, Sie möchten ein Bestellsystem erstellen, das diese Klassen verwendet. Bestellungen können einfache Produkte ohne Verpackung sowie vollgepackte Kartons enthalten. Wie würden Sie den Gesamtpreis einer solchen Bestellung ermitteln?



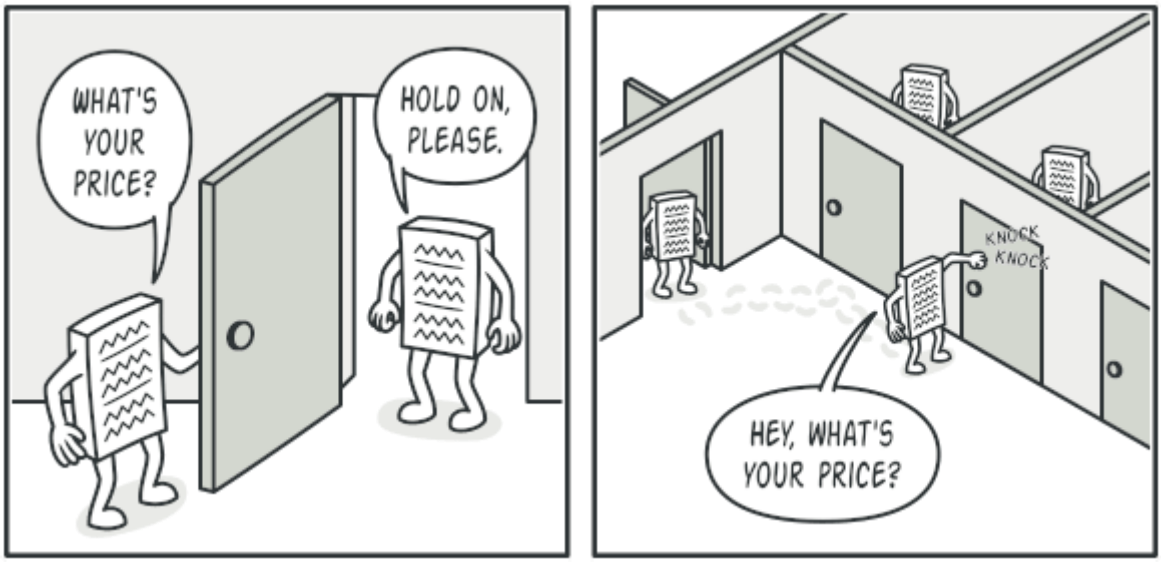
Eine Bestellung kann aus verschiedenen Produkten bestehen, die in Kartons verpackt sind, die wiederum in größeren Kartons verpackt sind usw. Die gesamte Struktur sieht aus wie ein umgedrehter Baum.

Sie könnten den direkten Ansatz versuchen: Packen Sie alle Kartons aus, gehen Sie alle Produkte durch und berechnen Sie dann die Summe. Das wäre in der Praxis machbar; in einem Programm ist es jedoch nicht so einfach wie das Ausführen einer Schleife. Sie müssen die Klassen, die Verschachtelungsebene der Kartons und andere wichtige Details im Voraus kennen `Products`. `Boxes`All dies macht den direkten Ansatz entweder zu umständlich oder sogar unmöglich.

Lösung

`Products`Das Composite-Muster schlägt vor, dass Sie mit und über eine gemeinsame Schnittstelle arbeiten `Boxes`, die eine Methode zur Berechnung des Gesamtpreises deklariert.

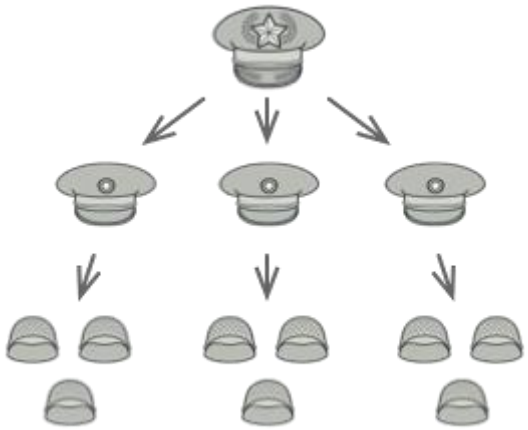
Wie würde diese Methode funktionieren? Für ein Produkt würde lediglich der Produktpreis zurückgegeben. Bei einer Schachtel würde jeder darin enthaltene Artikel durchgegangen, der Preis abgefragt und anschließend der Gesamtbetrag für diese Schachtel zurückgegeben. Handelte es sich bei einem dieser Artikel um eine kleinere Schachtel, würde auch deren Inhalt durchgegangen und so weiter, bis die Preise aller enthaltenen Komponenten berechnet sind. Eine Schachtel könnte sogar zusätzliche Kosten zum Endpreis hinzufügen, beispielsweise Verpackungskosten.



Mit dem Composite-Muster können Sie ein Verhalten rekursiv über alle Komponenten eines Objektbaums ausführen.

Der größte Vorteil dieses Ansatzes besteht darin, dass Sie sich nicht um die konkreten Objektklassen kümmern müssen, aus denen der Baum besteht. Sie müssen nicht wissen, ob es sich bei einem Objekt um ein einfaches Produkt oder eine komplexe Box handelt. Über die gemeinsame Schnittstelle können Sie sie alle gleich behandeln. Wenn Sie eine Methode aufrufen, geben die Objekte die Anfrage selbst im Baum weiter.

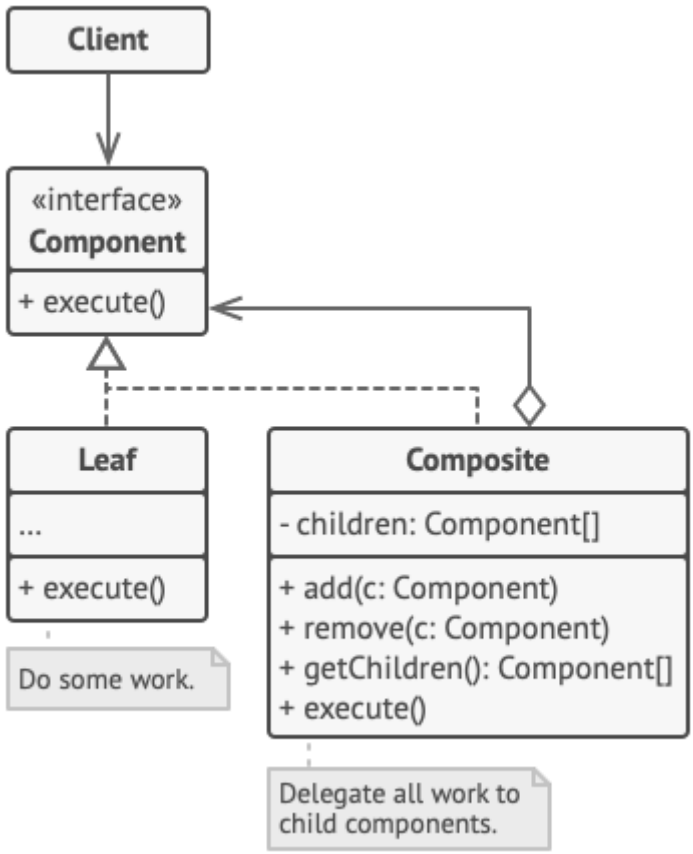
Analogie zur realen Welt



Ein Beispiel für eine militärische Struktur.

Die Armeen der meisten Länder sind hierarchisch strukturiert. Eine Armee besteht aus mehreren Divisionen; eine Division ist eine Gruppe von Brigaden, und eine Brigade besteht aus Zügen, die wiederum in Trupps unterteilt werden können. Ein Trupp ist eine kleine Gruppe echter Soldaten. Befehle werden an der Spitze der Hierarchie erteilt und an die einzelnen Ebenen weitergegeben, bis jeder Soldat weiß, was zu tun ist.

Struktur



1. Die **Komponentenschnittstelle** beschreibt Operationen, die sowohl einfachen als auch komplexen Elementen des Baums gemeinsam sind.
2. Das **Blatt** ist ein Grundelement eines Baums, das keine Unterelemente hat.

   Normalerweise erledigen Blattkomponenten letztendlich den Großteil der eigentlichen Arbeit, da sie niemanden haben, an den sie die Arbeit delegieren können.

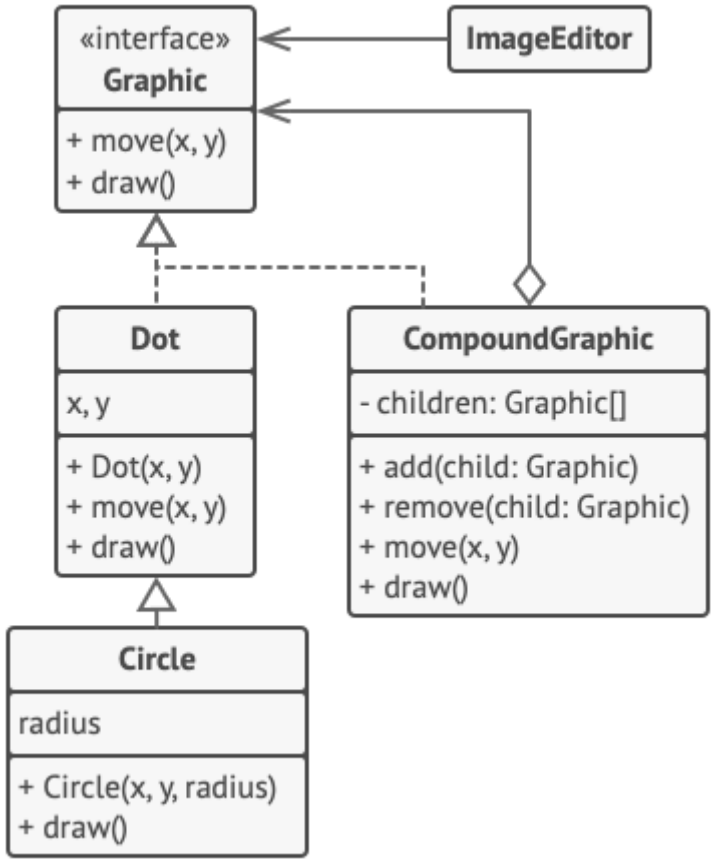
3. Der **Container** (auch *Composite* genannt) ist ein Element mit Unterelementen: Blättern oder anderen Containern. Ein Container kennt die konkreten Klassen seiner untergeordneten Elemente nicht. Er arbeitet mit allen Unterelementen ausschließlich über die Komponentenschnittstelle.

   Nach Erhalt einer Anfrage delegiert ein Container die Arbeit an seine Unterelemente, verarbeitet Zwischenergebnisse und gibt dann das Endergebnis an den Client zurück.

4. Der **Client** arbeitet mit allen Elementen über die Komponentenschnittstelle. Dadurch kann der Client sowohl mit einfachen als auch mit komplexen Elementen des Baums auf die gleiche Weise arbeiten.

Pseudocode

In diesem Beispiel können Sie mit dem **Composite** -Muster das Stapeln geometrischer Formen in einem grafischen Editor implementieren.



Das Beispiel des Editors für geometrische Formen.

Die `CompoundGraphic`Klasse ist ein Container, der eine beliebige Anzahl von Unterformen, einschließlich anderer zusammengesetzter Formen, enthalten kann. Eine zusammengesetzte Form verfügt über dieselben Methoden wie eine einfache Form. Anstatt jedoch selbst etwas auszuführen, leitet eine zusammengesetzte Form die Anforderung rekursiv an alle ihre untergeordneten Formen weiter und fasst das Ergebnis zusammen.

Der Clientcode arbeitet mit allen Formen über die einheitliche Schnittstelle, die allen Formklassen gemeinsam ist. Der Client erkennt daher nicht, ob er mit einer einfachen oder einer zusammengesetzten Form arbeitet. Er kann mit sehr komplexen Objektstrukturen arbeiten, ohne an konkrete Klassen gebunden zu sein, die diese Struktur bilden.

```
// The component interface declares common operations for both
// simple and complex objects of a composition.
interface Graphic is
    method move(x, y)
    method draw()

// The leaf class represents end objects of a composition. A
// leaf object can't have any sub-objects. Usually, it's leaf
// objects that do the actual work, while composite objects only
// delegate to their sub-components.
class Dot implements Graphic is
    field x, y

    constructor Dot(x, y) { ... }

    method move(x, y) is
        this.x += x, this.y += y

    method draw() is
        // Draw a dot at X and Y.

// All component classes can extend other components.
class Circle extends Dot is
    field radius

    constructor Circle(x, y, radius) { ... }

    method draw() is
        // Draw a circle at X and Y with radius R.

// The composite class represents complex components that may
// have children. Composite objects usually delegate the actual
// work to their children and then "sum up" the result.
class CompoundGraphic implements Graphic is
    field children: array of Graphic

    // A composite object can add or remove other components
    // (both simple or complex) to or from its child list.
    method add(child: Graphic) is
        // Add a child to the array of children.

    method remove(child: Graphic) is
        // Remove a child from the array of children.

    method move(x, y) is
        foreach (child in children) do
            child.move(x, y)
```

```
// A composite executes its primary logic in a particular
    // way. It traverses recursively through all its children,
    // collecting and summing up their results. Since the
    // composite's children pass these calls to their own
    // children and so forth, the whole object tree is traversed
    // as a result.
    method draw() is
        // 1. For each child component:
        //     - Draw the component.
        //     - Update the bounding rectangle.
        // 2. Draw a dashed rectangle using the bounding
        // coordinates.


// The client code works with all the components via their base
// interface. This way the client code can support simple leaf
// components as well as complex composites.
class ImageEditor is
    field all: CompoundGraphic

    method load() is
        all = new CompoundGraphic()
        all.add(new Dot(1, 2))
        all.add(new Circle(5, 3, 10))
        // ...

    // Combine selected components into one complex composite
    // component.
    method groupSelected(components: array of Graphic) is
        group = new CompoundGraphic()
        foreach (component in components) do
            group.add(component)
            all.remove(component)
        all.add(group)
        // All components will be drawn.
        all.draw()
```

Anwendbarkeit

Verwenden Sie das Composite-Muster, wenn Sie eine baumartige Objektstruktur implementieren müssen.
Das Composite-Muster bietet zwei grundlegende Elementtypen mit einer gemeinsamen Schnittstelle: einfache Blätter und komplexe Container. Ein Container kann sowohl aus Blättern als auch aus anderen Containern bestehen. Dadurch können Sie eine verschachtelte rekursive Objektstruktur erstellen, die einem Baum ähnelt.

Verwenden Sie das Muster, wenn der Clientcode sowohl einfache als auch komplexe Elemente einheitlich behandeln soll.
Alle durch das Composite-Muster definierten Elemente verfügen über eine gemeinsame Schnittstelle. Mithilfe dieser Schnittstelle muss sich der Client keine Gedanken über die konkrete Klasse der Objekte machen, mit denen er arbeitet.

So implementieren Sie

1. Stellen Sie sicher, dass das Kernmodell Ihrer App als Baumstruktur dargestellt werden kann. Versuchen Sie, es in einfache Elemente und Container zu zerlegen. Beachten Sie, dass Container sowohl einfache Elemente als auch andere Container enthalten können müssen.
2. Deklarieren Sie die Komponentenschnittstelle mit einer Liste von Methoden, die sowohl für einfache als auch für komplexe Komponenten sinnvoll sind.
3. Erstellen Sie eine Blattklasse zur Darstellung einfacher Elemente. Ein Programm kann mehrere verschiedene Blattklassen haben.
4. Erstellen Sie eine Containerklasse zur Darstellung komplexer Elemente. Stellen Sie in dieser Klasse ein Array-Feld zum Speichern von Verweisen auf Unterelemente bereit. Das Array muss sowohl Blätter als auch Container speichern können. Stellen Sie daher sicher, dass es mit dem Schnittstellentyp der Komponente deklariert ist.

   Bedenken Sie beim Implementieren der Methoden der Komponentenschnittstelle, dass ein Container den Großteil der Arbeit an Unterelemente delegieren soll.

5. Definieren Sie abschließend die Methoden zum Hinzufügen und Entfernen von untergeordneten Elementen im Container.

   Beachten Sie, dass diese Operationen in der Komponentenschnittstelle deklariert werden können. Dies würde das *Prinzip der Schnittstellentrennung* verletzen , da die Methoden in der Blattklasse leer wären. Der Client kann jedoch alle Elemente gleich behandeln, auch beim Erstellen des Baums.

Für und Wider

- Sie können bequemer mit komplexen Baumstrukturen arbeiten: Nutzen Sie Polymorphismus und Rekursion zu Ihrem Vorteil.

- *Offenes/Geschlossenes Prinzip* . Sie können neue Elementtypen in die App einführen, ohne den vorhandenen Code zu beschädigen, der jetzt mit dem Objektbaum funktioniert.

- Es kann schwierig sein, eine gemeinsame Schnittstelle für Klassen bereitzustellen, deren Funktionalität zu stark variiert. In bestimmten Szenarien müsste die Komponentenschnittstelle übermäßig generalisiert werden, was das Verständnis erschwert.

Beziehungen zu anderen Mustern

- **Sie können Builder** beim Erstellen komplexer **zusammengesetzter** Bäume verwenden, da Sie seine Konstruktionsschritte so programmieren können, dass sie rekursiv funktionieren.
- **Die Verantwortungskette** wird häufig in Verbindung mit **Composite** verwendet . In diesem Fall kann eine Blattkomponente, wenn sie eine Anforderung erhält, diese durch die Kette aller übergeordneten Komponenten bis zur Wurzel des Objektbaums weiterleiten.
- Sie können **Iteratoren verwenden, um zusammengesetzte** Bäume zu durchlaufen .
- Sie können **Visitor verwenden, um eine Operation für einen gesamten zusammengesetzten** Baum auszuführen .
- Sie können gemeinsam genutzte Blattknoten des **zusammengesetzten** Baums als **Flyweights** implementieren , um RAM zu sparen.
- **Composite** und **Decorator** haben ähnliche Strukturdiagramme, da beide auf rekursiver Komposition basieren, um eine unbegrenzte Anzahl von Objekten zu organisieren.

  Ein *Decorator* ähnelt einem *Composite* , hat aber nur eine untergeordnete Komponente. Es gibt einen weiteren wesentlichen Unterschied: *Der Decorator* fügt dem umschlossenen Objekt zusätzliche Verantwortlichkeiten hinzu, während *das Composite* lediglich die Ergebnisse seiner untergeordneten Komponenten zusammenfasst.

  Die Muster können jedoch auch zusammenarbeiten: Sie können *Decorator* verwenden , um das Verhalten eines bestimmten Objekts im *Composite*- Baum zu erweitern.

- **Designs, die häufig Composite** und **Decorator** verwenden , profitieren oft von der Verwendung von **Prototype** . Durch die Anwendung des Musters können Sie komplexe Strukturen klonen, anstatt sie von Grund auf neu zu erstellen.